

第9章 并发与异常

建议学时:5-8 小时 | 难度:★★★★★ | 读者背景:已掌握 C 语言

本章学习目标

- 理解单线程事件循环:一个进程一条 JS 线程,并发靠"快速切换任务"
- 彻底分清宏任务与微任务,能推演 setTimeout/Promise 混合顺序题
- 掌握 try/catch/finally 与 throw,理解与 C 的错误处理对照
- 掌握 Promise 全套:状态机、链式、all/race/allSettled/any
- 掌握 async/await:语法糖本质、错误传播、并发执行
- 理解 throw 与 Promise 错误的传播路径,掌握 unhandledRejection 防御
- 理解异步竞态与互斥锁模拟、防抖节流,认识 worker_threads/Cluster

从 C 到 JavaScript:本章主题如何迁移

C 的并发是"多线程 + 锁":pthread_create 开线程,互斥锁保护共享变量,条件变量协调等待。JS 走的是完全不同的路:单线程事件循环——一个进程只有一条 JS 线程,同时处理上万个连接靠的不是并行,而是"每个任务都很快,做完就切换到下一个"。没有线程共享变量的竞态,锁与条件变量在业务代码里几乎消失,换来的是异步回调的思维。

这套模型有两个必须理解的心智支点。第一,同步代码永远先执行完,异步代码永远排在后面:setTimeout 的 0 毫秒不是"立即",而是"当前同步代码之后"。第二,异步任务内部还分两级:微任务(Promise)比宏任务(setTimeout/I/O)优先。这两条规则能推导出所有"输出顺序题"。

异常方面,JS 的 try/catch 与 C++/Java 类似,但有一条关键差异:异步函数的 throw 不会冒泡到调用处——它变成 Promise 的拒绝,必须用 .catch 或 await 捕获。C 的 errno/返回值约定在 JS 里被"抛异常 + Promise 拒绝"二分替代,这条传播路径必须形成肌肉记忆。

最后,JS 不是没有多线程:worker_threads 可以开真线程(各自独立事件循环),Cluster 可以多进程共享端口。但生产的第一选择永远是"单线程 + 异步",把 CPU 密集任务扔给 worker,把并发流量交给 Cluster/容器横向扩展。读本章时,请带着"事件循环怎么转"这个问题逐节验证。

前置知识自查

- C 的异常域:没有 try/catch,错误靠 errno/返回值/assert
- C 的 pthread:线程、互斥锁、条件变量、join
- 第4章回调与闭包(本章异步的基础)
- 第7章 async fs 的用法

本章学习路线

1. **第一阶段(事件循环)**:读 §9.1,把 setTimeout 顺序题亲手跑一遍,建立"队列模型"
2. **第二阶段(异常)**:读 §9.2,理解同步异常与异步错误的区别
3. **第三阶段(Promise/async,本章核心)**:读 §9.3–§9.5,每个示例在 REPL 跑通,重点练错误传播
4. **第四阶段(并发实践)**:读 §9.6–§9.7,完成章末实验"任务队列处理器"
5. **验收标准**:不看资料,能推演三段 setTimeout/Promise 混排代码的输出,并解释微任务优先原因

本章知识导图

- 第9章 并发与异常
 - §9.1 单线程事件循环:宏任务/微任务、顺序题
 - §9.2 异常处理:throw / try / catch / finally
 - §9.3 Promise 全解:状态机、链式、并行组合
 - §9.4 async/await:语法糖、并发、错误传播
 - §9.5 错误传播:unhandledRejection 与全局防御
 - §9.6 异步竞态:互斥模拟、防抖、节流
 - §9.7 worker_threads 与 Cluster

§9.1 单线程事件循环:宏任务与微任务

直觉先行:一条线程,两个队列

JS 运行时有一条主线程和一个事件循环。同步代码直接执行;异步操作(setTimeout、I/O、Promise)登记到队列,等同步代码全部执行完,事件循环才开始取队列任务。队列分两级:微任务队列(Promise.then/catch、queueMicrotask)永远先于宏任务队列(setTimeout、setInterval、I/O 回调)。每执行完一个宏任务,先把微任务队列清空,再取下一个宏任务。

```
// 经典顺序题:先背下"同步 > 微任务 > 宏任务"
console.log('1 同步');
setTimeout(() => console.log('5 宏任务'), 0);
Promise.resolve().then(() => console.log('3 微任务'));
console.log('2 同步');
// 输出顺序:1 2 3 5
// 解释:同步代码(1、2)先全执行;微任务(3)先于宏任务(5)

// 进阶:微任务里注册的宏任务排在后面
setTimeout(() => console.log('A 宏任务'), 0);
Promise.resolve()
  .then(() => console.log('B 微任务'))
  .then(() => setTimeout(() => console.log('C 宏任务(微任务中注册)'), 0));
// 输出:A B C
// 解释:第一轮:A(宏)→清空微任务(B)→ 第二轮:C
// 若 B 不注册 C,则 C 与 A 同轮,顺序看注册先后

// 综合题(面试高频):
setTimeout(() => console.log('t1'), 0);
setTimeout(() => console.log('t2'), 0);
Promise.resolve().then(() => console.log('p1'));
Promise.resolve().then(() => console.log('p2'));
console.log('sync');
// 输出:sync p1 p2 t1 t2
```

★ 必须记住的调度规则

① 同步代码永不被打断:一段同步代码(含循环)执行完,才轮到任何异步任务;② 微任务优先:每轮宏任务后清空微任务队列;③ `setTimeout(0)` 是"最早的下一个宏任务",不是立即;④ `await` 本质是 `.then` 的语法糖,同样走微任务。

9.1.1 为什么单线程能支撑高并发

```
const http = require('node:http');

// 一个进程,一条线程,同时服务海量连接:
// 每个请求的处理都是"小任务",I/O 等待期间线程去处理别的请求
const server = http.createServer((req, res) => {
  // 模拟异步 I/O:setTimeout 期间,线程可以去处理其他请求
  setTimeout(() => {
    res.end('ok');
  }, 10);
});
server.listen(9000);

// 对比 C 的阻塞式:一个连接一个线程,万级连接需要万级线程;
// Node 单线程 + 非阻塞 I/O:万级连接共享一条线程,内存开销小一个数量级
// 代价:CPU 密集任务会卡住所有连接(见 §9.7 worker_threads)
```

§9.2 异常处理:throw / try / catch / finally

对比项	C	JS
错误载体	errno / 返回值(-1)/ NULL	异常对象 throw;异步用 Promise 拒绝
抛出	return -1; errno = ...	throw new Error('...')
捕获	if (x == -1) 逐层检查	try { } catch (err) { }
清理	goto err: 手动清理	finally 保证执行
未捕获	静默错误,行为未定义	进程崩溃(同步)/ unhandledRejection(异步)

```
// 基础语法:与 C++/Java 的 try/catch 同构
function divide(a, b) {
  if (b === 0) throw new Error('除数不能为 0'); // throw 任何值,规范用 Error
  return a / b;
}

try {
  const r = divide(10, 0);
  console.log('结果:', r);
} catch (err) {
  console.log('捕获:', err.message); // 捕获:除数不能为 0
} finally {
  console.log('finally 总是执行'); // 无论成败
}

// 错误类型:Error 家族
try {
  JSON.parse('{bad}');
} catch (err) {
  console.log(err.name); // SyntaxError
  console.log(err.message); // Unexpected token...
}

try { null.x; } catch (err) { console.log(err.name); } // TypeError
try { missingVar; } catch (err) { console.log(err.name); } // ReferenceError
console.log(new RangeError('越界').name); // RangeError

// 自定义错误:继承 Error
class ConfigError extends Error {
  constructor(key) {
    super(`配置缺失:${key}`);
    this.name = 'ConfigError';
    this.key = key; // 附带业务字段
  }
}

try {
  throw new ConfigError('port');
} catch (err) {
  if (err instanceof ConfigError) {
    console.log('业务错误,字段:', err.key, ' | 消息:', err.message);
  }
}

// finally 与 return 的先后:finally 先执行,再返回
function f() {
  try { return 'try 的返回值'; }
  finally { console.log('finally 在 return 前执行'); }
}

console.log(f());
```

⚠️ 常见误区 9.1:只 catch 不处理,或吞掉错误

`catch (err) {}` 空捕获等于把错误埋掉,排查时只剩"功能神秘失效"。生产规则:catch 后必须做三选一——① 记录日志再抛出/拒绝;② 降级到默认值并告警;③ 向上传播。绝不空 catch。另外 `catch` 的 `err` 是局部变量,外部同名变量不受影响。

§9.3 Promise 全解:状态机与链式

9.3.1 为什么需要 Promise:回调地狱的进化

```
// 回调嵌套(老代码,可读性差):
function readChain() {
  const fs = require('node:fs');
  fs.readFile('a.txt', 'utf8', (err, a) => {
    if (err) return console.error(err);
    fs.readFile('b.txt', 'utf8', (err, b) => {
      if (err) return console.error(err);
      fs.writeFile('c.txt', a + b, (err) => {
        if (err) return console.error(err);
        console.log('完成(回调嵌套)');
      });
    });
  });
}
// readChain(); // 需要先创建 a.txt b.txt

// Promise 化:new Promise 包装异步任务
const fs = require('node:fs');
function readFileP(path) {
  return new Promise((resolve, reject) => {
    fs.readFile(path, 'utf8', (err, data) => {
      if (err) reject(err);      // 失败 → 拒绝
      else resolve(data);       // 成功 → 兑现
    });
  });
}
```

9.3.2 状态机:三个状态,一次转移

```
// Promise 只有三种状态:pending(进行中)→ fulfilled(兑现)/ rejected(拒绝)
// 状态只能转移一次,不可逆—这保证了 .then 的结果确定性

const p = new Promise((resolve, reject) => {
  // 异步任务在这里启动
  setTimeout(() => resolve('成功值'), 100);
});
console.log('初始状态:promise(状态不可直接读)');
p.then((value) => console.log('兑现:', value)); // 兑现:成功值

// 拒绝的两种写法:reject() 或 throw(在 executor 中)
const p2 = new Promise((resolve, reject) => {
  throw new Error('executor 中抛错 = 拒绝');
});
p2.catch((err) => console.log('捕获:', err.message));

// 链式:then 返回新 Promise,自动"扁平化"
Promise.resolve(1)
  .then((n) => n + 1) // 返回普通值 → 自动包装
  .then((n) => Promise.resolve(n * 10)) // 返回 Promise → 展开等待
  .then((n) => console.log('链式结果:', n)) // 链式结果:20
  .catch((err) => console.error('任一环出错都到这里:', err.message));

// finally:无论成败都执行(清理资源),不改变传递的值
Promise.resolve('x')
  .finally(() => console.log('finally 清理'))
  .then((v) => console.log('值仍传递:', v));
```

⚠ 常见误区 9.2:then 里抛错没 catch,或忘记 return

① 链式必须 **return**: `p.then(() => { return fetchB(); })` 少了 `return`,链就断成"undefined 继续",后续拿不到 B 的结果;② 链尾必须挂 `catch` 或由调用方 `await`,否则拒绝无人处理会触发 `unhandledRejection`(见 §9.5)。ESLint 的 `promise` 规则组专门抓这两类。

9.3.3 并行组合:all / race / allSettled / any

```
// Promise.all:全部成功才成功(任何一个失败整体失败)——并行 I/O 首选
const tasks = [Promise.resolve(1), Promise.resolve(2), Promise.resolve(3)];
Promise.all(tasks).then(([a, b, c]) => console.log('all:', a + b + c)); // 6

// 生产场景:并发请求
async function fetchAll() {
  const urls = ['/api/a', '/api/b', '/api/c'];
  const results = await Promise.all(urls.map((u) => fetchJson(u)));
  return results;
}
async function fetchJson(u) { return { url: u, data: 1 }; } // 示意

// Promise.allSettled:全部完成(含失败),逐个看结果——批量任务不互相连坐
Promise.allSettled([Promise.resolve(1), Promise.reject(new Error('失败'))])
  .then((results) => {
    for (const r of results) {
      console.log(r.status, r.status === 'fulfilled' ? r.value : r.reason.message);
    }
  });

// Promise.race:谁先完成用谁(超时实现)
const slow = new Promise((res) => setTimeout(() => res('慢任务'), 1000));
const fast = new Promise((res) => setTimeout(() => res('快任务'), 100));
Promise.race([slow, fast]).then((v) => console.log('race 胜者:', v)); // 快任务

// Promise.any:第一个成功者;全失败才拒绝(ES2021)
Promise.any([Promise.reject(new Error('a 失败')), Promise.resolve('b 成功')])
  .then((v) => console.log('any:', v)); // b 成功
```

§9.4 async/await:语法糖与错误传播

async 函数 = 返回 Promise 的函数;await 表达式 = 取出 Promise 的值。它把“链式回调”写成同步风格,是 ES2017 以来 JS 异步的第一选择。**async 函数里 throw 等价于返回 rejected Promise**,await 遇到拒绝就在那行抛出,可用 try/catch 捕获——这正是“异步异常同步化”的核心。


```

// async 函数总是返回 Promise
async function hello() {
  return '你好'; // 自动包装成 Promise
}
console.log(hello() instanceof Promise); // true
hello().then((v) => console.log(v)); // 你好

// await 等待:与 then 等价但可读性更高
async function demo() {
  const a = await Promise.resolve(10);
  const b = await Promise.resolve(20);
  console.log('await 结果:', a + b); // 30
}
demo();

// 错误传播:async 里 throw = 返回 rejected Promise
async function risky() {
  throw new Error('async 里抛错');
}
risky().catch((err) => console.log('外部捕获:', err.message));

// 错误传播:await 拒绝在行内抛出,可以被 try/catch 捕获
async function withTry() {
  try {
    await Promise.reject(new Error('被拒绝'));
  } catch (err) {
    console.log('await 行内捕获:', err.message);
  }
}
withTry();

// await 并发:多个独立异步任务要并行,不要串行 await
async function parallel() {
  // ❌ 串行:总耗时 300ms
  const a = await delay(100, 'A');
  const b = await delay(100, 'B');
  console.log('串行完成:', a, b);

  // ✅ 并行:总耗时 100ms(先启动,再统一 await)
  const [c, d] = await Promise.all([delay(100, 'C'), delay(100, 'D')]);
  console.log('并行完成:', c, d);
}
function delay(ms, v) {
  return new Promise((res) => setTimeout(() => res(v), ms));
}
parallel();

```

⚠️ 常见误区 9.3:在循环里串行 await 拖慢整体

for (const u of urls) { const r = await fetch(u); } 是串行:100 个请求 100 秒。无依赖时改为 `await Promise.all(urls.map(u => fetch(u)))` ;有依赖(每个请求需要前一个结果)才保留串行。这是生产性能评审的必查项。

§9.5 错误传播:unhandledRejection 与全局防御

```
// 未处理的 Promise 拒绝:默认 Node 会打警告并(20+)可能崩溃进程
// Promise.reject(new Error('没人管我')); // UnhandledPromiseRejection

// 全局兜底:生产必须挂,把漏网错误打日志(但绝不能靠它替代正确 catch)
process.on('unhandledRejection', (reason, promise) => {
  console.error('未处理的 Promise 拒绝:', reason?.message ?? reason);
  // 生产:记录告警;进程是否退出视业务而定(默认退出更安全)
});
process.on('uncaughtException', (err) => {
  console.error('未捕获异常:', err.message);
  // 生产:记录后优雅退出(状态可能已损坏)
});

// EventEmitter 的 error 事件:emit('error') 没人监听会抛异常
const { EventEmitter } = require('node:events');
const em = new EventEmitter();
em.on('error', (err) => console.log('emitter 错误:', err.message));
em.emit('error', new Error('业务错误'));

// 错误的正确传播模板:函数返回 Promise,失败必 reject
async function getUser(id) {
  if (typeof id !== 'number' || id <= 0) {
    throw new TypeError(`非法 id:${id}`); // 参数错误立即拒绝
  }
  const user = await dbFind(id); // 数据库错误自然上抛
  return user ?? null; // 业务"未找到"用 null,不是异常
}
async function dbFind(id) { return { id }; }
getUser(-1).catch((e) => console.log('传播到调用方:', e.message));

// 错误边界经验:参数校验错误 → throw(立即失败);
// 业务"没找到" → 返回 null/undefined;系统错误 → 让异常沿链传播,由调用方决定
```

★ 必须记住的传播对照

同步代码: throw → 最近的 try/catch 或进程崩溃 ;异步回调: 回调风格错误走 err 参数(如 fs.readFile 的 (err, data));Promise 域: throw/reject → .catch → 链尾或 unhandledRejection ;async/await: throw → await 行抛出 → 外层 try/catch 。四域四规则,记熟后错误排查不再靠猜。

§9.6 异步竞态与同步原语

9.6.1 异步竞态:检查与使用之间的缝隙

```
// 竞态示例:并发请求同一资源,晚到的结果覆盖早到的
let current = 'v0';
async function loadVersion() {
  // 两个请求先后发出,返回顺序不定
  const r1 = load('/api/v?t=1');
  const r2 = load('/api/v?t=2');
  const [v1, v2] = [await r1, await r2];
  console.log('竞态风险:最终值取决于返回顺序');
}

function load(url) {
  return new Promise((res) => {
    const delay = url.includes('t=1') ? 200 : 50;
    setTimeout(() => res(`数据(${delay}ms)`), delay);
  });
}

// 经典解法:请求序号校验(丢弃过期响应)
let requestSeq = 0;
async function safeLoad() {
  const mySeq = ++requestSeq;           // 本次请求序号
  const result = await load('/api/data');
  if (mySeq !== requestSeq) {
    console.log('响应已过期,丢弃');    // 有更新的请求在途
    return null;
  }
  return result;                       // 最新请求才生效
}
```

9.6.2 互斥锁模拟:串行化异步临界区

```
// 单线程也有"临界区":await 会出让控制权,期间状态可能被改
// 用 Promise 链实现"互斥锁"(串行队列):
class Mutex {
  #queue = Promise.resolve();
  #lock() {
    let release;
    const next = new Promise((res) => { release = res; });
    const prev = this.#queue;
    this.#queue = this.#queue.then(() => next);
    return prev.then(() => release);
  }
  async run(fn) {
    const release = await this.#lock();
    try {
      return await fn();
    } finally {
      release(); // 无论成败都解锁
    }
  }
}

// 使用:多个异步任务排队执行,临界区不交错
const mtx = new Mutex();
let counter = 0;
async function inc(n) {
  for (let i = 0; i < n; i++) {
    await mtx.run(async () => { // 临界区串行
      const cur = counter;      // 读
      await delay(0);           // 模拟异步间隙
      counter = cur + 1;        // 写
    });
  }
}

function delay(ms) { return new Promise((r) => setTimeout(r, ms)); }

async function main() {
  await Promise.all([inc(50), inc(50)]);
  console.log('加锁后计数:', counter); // 100,没有丢失更新
}

main();
```

9.6.3 防抖与节流:高频事件的两种整形

```
// 防抖(debounce):停止触发后 N 毫秒才执行一次(输入联想、搜索框)
function debounce(fn, wait = 300) {
  let timer = null;
  return (...args) => {
    clearTimeout(timer); // 重置
    timer = setTimeout(() => fn(...args), wait);
  };
}

const debouncedSearch = debounce((q) => console.log('搜索:', q));
debouncedSearch('a');
debouncedSearch('ab');
debouncedSearch('abc'); // 只会输出一:搜索 abc

// 节流(throttle):N 毫秒内最多执行一次(滚动、拖拽、限频上报)
function throttle(fn, wait = 200) {
  let last = 0;
  return (...args) => {
    const now = Date.now();
    if (now - last >= wait) {
      last = now;
      fn(...args);
    }
  };
}

const throttledLog = throttle((msg) => console.log('节流:', msg));
for (let i = 0; i < 5; i++) throttledLog(i); // 只输出一:节流: 0
```

§9.7 worker_threads 与 Cluster:真并行的两条路

```
// worker_threads:CPU 密集任务的真并行(各自独立事件循环,通过消息通信)
const { Worker } = require('node:worker_threads');

function runInWorker(code) {
  return new Promise((resolve, reject) => {
    const worker = new Worker(code, { eval: true });
    worker.on('message', resolve);
    worker.on('error', reject);
    worker.on('exit', (code2) => {
      if (code2 !== 0) reject(new Error(`worker 退出码 ${code2}`));
    });
  });
}

// 大数计算放 worker,主线程事件循环不卡
async function heavyCompute() {
  const result = await runInWorker(`
    const { parentPort } = require('node:worker_threads');
    let sum = 0;
    for (let i = 0; i < 1e8; i++) sum += i;    // 纯 CPU 密集
    parentPort.postMessage(sum);
  `);
  console.log('worker 计算结果:', result);
}

// heavyCompute(); // 与主线程并行,主线程仍能响应请求

// Cluster:多进程共享端口(每个进程一个事件循环,利用多核 CPU)
// node --experimental-... 老 API;现代:用 cluster 模块分派
// if (cluster.isPrimary) { fork() 若干 worker } else { 启动 http 服务器 }
// 一句话:单机多核利用 = Cluster(进程)或 worker_threads(线程);
// 横向扩展 = 容器多副本 + 负载均衡,这是生产主流
```

工程建议 9.1:并发模型选型决策树

I/O 密集(网络/文件/数据库)→ 单线程异步足够,万级并发无压力;CPU 密集(加密、图像、大数运算)→ worker_threads 分担,数量 = CPU 核数 - 1;需要独立故障域/多核吞吐 → Cluster 或容器多副本;跨进程消息(多实例)→ Redis/消息队列。先问"卡的是 I/O 还是 CPU",再选工具,不要一上来就上多进程。

本章速查表

主题	速查要点
模型	单线程事件循环;同步代码先执行完,异步排队
任务分级	微任务(Promise.then/await)优先于宏任务(setTimeout/I/O)
setTimeout(0)	最早的下一个宏任务,不是立即
高并发原理	非阻塞 I/O + 快速切换;CPU 密集会卡事件循环
throw	任意值可抛;生产统一 new Error(...)
catch/finally	catch 后必须处理/记录/重抛;finally 保证清理
Promise 状态	pending → fulfilled/rejected,只能转移一次
链式	then 返回新 Promise;必须 return 才能继续传值;链尾挂 catch
组合	all 全成、allSettled 全落、race 先到、any 先成
async/await	async 返回 Promise;await 取结果/在行内抛错;throw = reject
并发等待	独立任务用 Promise.all 并行,不要串行 await
全局兜底	unhandledRejection / uncaughtException 挂日志与退出策略
异步竞态	序号校验丢弃过期响应
互斥模拟	Promise 链串行化临界区(生产多用库:async-mutex)
防抖/节流	debounce 停后执行;throttle 限频执行
真并行	worker_threads(CPU 密集);Cluster/容器副本(多核/横向)

最小必会清单

- 能推演"同步、微任务、宏任务"三段混排的输出顺序,并解释原因
- 能说出 Promise 三状态与"只能转移一次"的含义
- 能写出链式 then + 链尾 catch,并说明漏 return 的后果
- 能说出 all/allSettled/race/any 四个组合的语义差异
- 能写出 async 函数中 throw 与 await 拒绝的两种捕获方式
- 能说出"循环里串行 await"的优化方法
- 能写出 unhandledRejection 全局兜底并说明为何不能依赖它
- 能实现一个 Promise 链互斥锁
- 能区分防抖与节流并各写一个实现
- 能说出 worker_threads 与 Cluster 的适用场景

自测题

Q 1. 写出输出顺序:`console.log('a');` `setTimeout(() => console.log('b'), 0);` `Promise.resolve().then(() => console.log('c'));` `console.log('d');`

✅ 参考答案

a d c b。同步 a、d 先执行;微任务 c 优先于宏任务 b。

Q 2. `async` 函数里 `throw new Error('x')` 会发生什么?调用方如何捕获?

✅ 参考答案

函数返回一个 rejected Promise。捕获:调用方 `await` 后 `try/catch`,或 `.catch()`。

Q 3. `Promise.all` 与 `Promise.allSettled` 的差异?批量下载任务该用哪个?

✅ 参考答案

`all` 只要一个失败就整体拒绝,拒绝即放弃其余结果;`allSettled` 等全部完成,逐条看 `status`。批量下载用 `allSettled`(单条失败不影响统计),全部成功才继续的流程用 `all`。

Q 4. 为什么 `for` 循环里串行 `await` 拖慢性能?怎么改?

✅ 参考答案

每个 `await` 等前一个完成才发下一个,总耗时是各请求之和。无依赖改为 `await Promise.all(arr.map(fn))`,总耗时约等于最慢的单个。

Q 5. 同步 `throw` 未捕获与 `Promise` 拒绝未处理,后果分别是什么?

✅ 参考答案

同步未捕获走 `uncaughtException`,进程崩溃(可兜底记录后退出);`Promise` 拒绝未处理走 `unhandledRejection`,Node 默认告警且 20+ 版本默认使进程退出。

Q 6. 防抖与节流的区别?搜索框联想与滚动加载各用哪个?

✅ 参考答案

防抖:连续触发只执行最后一次(停止后计时);节流:固定间隔最多执行一次。搜索联想用防抖,滚动加载用节流。

章末实验题:任务队列处理器(生产者-消费者)

实验 9:任务队列处理器(生产者-消费者)

实验目标:实现一个并发受控的任务队列:生产者投递任务,消费者按并发上限处理,失败自动重试,全部完成输出统计——覆盖全章知识点。

实验要求:

- 任务 1(覆盖:Promise 状态、链式):任务队列用 Promise 链维护;每个任务包装成 Promise
- 任务 2(覆盖:async/await、错误传播):消费者用 async 函数处理任务,内部 throw 上抛
- 任务 3(覆盖:异常、自定义错误):定义 TaskError(带任务 id),重试超过上限时抛出并记录
- 任务 4(覆盖:allSettled、race):汇总统计用 allSettled;超时控制用 race(任务与计时器赛跑)
- 任务 5(覆盖:事件循环顺序):演示队列先同步入队、异步消费的调度(输出顺序体现宏/微任务)
- 任务 6(覆盖:互斥模拟):共享"成功计数"用 Mutex 保护,验证计数无丢失
- 任务 7(覆盖:防抖/节流):附加实现 debounce/throttle 各一,并验证调用次数被整形
- 任务 8(覆盖:unhandledRejection 防御):挂全局兜底,并故意产生一个漏网拒绝观察兜底输出

实验环境:Node.js 20+,运行 `node taskqueue.js`。

第一步 **写队列类:**TaskQueue{ push(task), concurrency 限制, 结果回调 }

第二步 **写消费者:**每个任务 fn 返回 Promise;并发数 = 同时运行上限;完成一个补一个

第三步 **后组装演示:**30 个随机任务(部分故意失败),重试 2 次,输出统计与顺序观察

✅ 参考答案要点

核心:TaskQueue 并发控制 + 重试装饰 + allSettled 汇总 + Mutex 计数 + 全局兜底。约 150 行,覆盖全章知识点。

```
// taskqueue.js — Node.js 20+ 运行:node taskqueue.js
'use strict';

// 1. 自定义错误(覆盖:Error 继承、异常体系)
class TaskError extends Error {
  constructor(taskId, msg) {
    super(`任务 ${taskId} ${msg}`);
    this.name = 'TaskError';
    this.taskId = taskId;
  }
}

// 2. 互斥锁(覆盖:Promise 链串行化临界区)
class Mutex {
  #tail = Promise.resolve();
  async run(fn) {
    let release;
    const gate = new Promise((res) => { release = res; });
    const prev = this.#tail;
    this.#tail = prev.then(() => gate);
    await prev;
    try {
      return await fn();
    } finally {
      release();
    }
  }
}

// 3. 任务队列(覆盖:并发控制、Promise、错误传播)
class TaskQueue {
  #running = 0;
  #results = [];
  #mutex = new Mutex();
  constructor(concurrency = 3, { retries = 1, timeoutMs = 2000 } = {}) {
    this.concurrency = concurrency;
    this.retries = retries;
    this.timeoutMs = timeoutMs;
  }

  async #runOne(task, id) {
    let attempt = 0;
    for (;;) {
      attempt++;
      try {
        // 超时控制:与计时器赛跑(覆盖:Promise.race)
        const result = await Promise.race([
          task(),
          new Promise((_, rej) =>
            setTimeout(() => rej(new TaskError(id, `超时(${this.timeoutMs}ms)`)), this.timeoutMs)
          ),
        ]);
        return { ok: true, id, result, attempt };
      } catch (err) {
        if (attempt > this.retries) {

```

```

        return { ok: false, id, error: err, attempt }; // 重试耗尽
    }
    await delay(10 * attempt); // 退避
}
}
}

// 入队 + 并发消费(覆盖:async/await、共享计数)
async run(tasks) {
    const entries = tasks.map((t, i) => ({ task: t, id: i + 1 }));
    const workers = Array.from(
        { length: Math.min(this.concurrency, entries.length) },
        () => this.#worker(entries)
    );
    await Promise.all(workers);
    return this.#results;
}

async #worker(entries) {
    while (entries.length > 0) {
        const e = entries.shift(); // 取任务
        const r = await this.#runOne(e.task, e.id);
        await this.#mutex.run(async () => { // 临界区:计数
            this.#results.push(r);
        });
    }
}

// 4. 全局兜底(覆盖:unhandledRejection 防御)
process.on('unhandledRejection', (reason) => {
    console.error('【兜底】未处理的 Promise 拒绝:', reason?.message ?? reason);
});

// 5. 工具(覆盖:delay、随机)
function delay(ms) { return new Promise((r) => setTimeout(r, ms)); }
function makeTask(i) {
    return async () => {
        await delay(20 + Math.random() * 80); // 模拟耗时
        if (Math.random() < 0.25) {
            throw new TaskError(i, '随机失败'); // 25% 失败
        }
        return `任务${i}结果`;
    };
}

// 6. 防抖/节流(覆盖:高频事件整形)
function debounce(fn, wait) {
    let timer = null;
    return (...args) => {
        clearTimeout(timer);
        timer = setTimeout(() => fn(...args), wait);
    };
}

function throttle(fn, wait) {

```

```

let last = 0;
return (...args) => {
  const now = Date.now();
  if (now - last >= wait) { last = now; fn(...args); }
};
}

// 7. 事件循环顺序演示(覆盖:宏任务/微任务)
function orderDemo() {
  console.log('- 顺序演示 -');
  console.log('1 同步');
  setTimeout(() => console.log('4 宏任务'), 0);
  Promise.resolve().then(() => console.log('2 微任务'));
  queueMicrotask(() => console.log('3 queueMicrotask'));
}

// 8. 主流程(覆盖:allSettled 汇总、统计)
async function main() {
  orderDemo();

  const tasks = Array.from({ length: 30 }, (_, i) => makeTask(i));
  const queue = new TaskQueue(4, { retries: 2, timeoutMs: 500 });
  const results = await queue.run(tasks);

  const okCount = results.filter((r) => r.ok).length;
  const failCount = results.length - okCount;
  console.log('- 队列统计 -');
  console.log(`总任务:${results.length} | 成功:${okCount} | 失败:${failCount}`);
  const retried = results.filter((r) => r.attempt > 1).length;
  console.log(`重试后成功:${retried} 个(含 attempt>1)`);

  const summaries = await Promise.allSettled(
    results.filter((r) => !r.ok).map((r) => Promise.reject(r.error))
  );
  console.log('失败的拒绝统计:', summaries.length);

  // 防抖/节流验证
  let dbg = 0;
  const db = debounce(() => { dbg++; }, 50);
  for (let i = 0; i < 10; i++) db();
  await delay(80);
  console.log('防抖:连续 10 次调用,实际执行', dbg, '次');

  // 漏网拒绝(演示兜底)
  // Promise.reject(new Error('故意漏网的拒绝'));
}

main().catch((err) => {
  console.error('主流程异常:', err.message);
  process.exitCode = 1;
});

```

设计说明:TaskQueue 用固定数量 worker 循环取任务,天然实现并发上限;单任务超时时用 Promise.race 与计时器赛跑;重试带指数退避(10ms × attempt),重试耗尽返回 {ok:false} 不炸队列;成功/失败统计用 Mutex 保护共享数

组(演示异步临界区);orderDemo 直观展示同步→微任务→宏任务顺序;防抖/节流验证调用次数整形;unhandledRejection 兜底注释掉的漏网拒绝可打开观察。修改 concurrency 从 1 到 8 对比总耗时,可以直观感受并发度对吞吐的影响。

下一章预告:第10章 泛型与元编程 —— JS 没有静态类型参数,TypeScript 泛型指明方向;Proxy/Reflect 的元编程能力是本章主场。